

Escrow Pay

Project objective and scope, for CLOCK IN, the Solana Mobile hackathon by RadiantsDAO

Builder	Kris, Speak Technology Ltd, Lagos, Nigeria
Stack	Flutter (mobile), Anchor and Rust (onchain program)
Base project	Adapted from Sellit, an existing P2P escrow resale marketplace
Network	Solana Devnet during development, targeting mainnet for submission

Objective

Build a mobile first, onchain escrow app for the Solana dApp Store, where a seller and buyer settle a resale transaction without trusting each other directly. Funds sit in a program controlled account until the buyer confirms receipt, at which point the program releases them to the seller. The goal for the hackathon is a tight, fully working demo, not a broad feature set.

What we are building

Actors

Seller and Buyer. No admin role, no third party arbiter for v1.

Core object

An Escrow Account, a program derived address (PDA) on Solana that holds the buyer's funds until release conditions are met. Neither party controls this account directly; the onchain program governs when funds move.

User flow

- 1 Seller creates a listing in app (item name, price in USDC or SOL), which generates a QR code encoding the escrow parameters.
- 2 Buyer scans the QR, reviews the details, connects their wallet, and funds the escrow. This calls the program's `initialize_escrow` and `deposit` instructions.
- 3 Item changes hands in person.
- 4 Buyer taps confirm receipt, which calls `confirm_receipt` and releases funds to the seller.
- 5 If something is wrong, the buyer can trigger a refund path before confirming, which returns funds to themselves.

States

Created → Funded → Released, or Funded → Refunded. Four states total. A dispute or arbitration state is a stretch goal, not part of v1.

Design direction

The app should feel futuristic and worthy of a blockchain product. Priorities in order: motion and feedback first, since that is what a demo actually shows in ninety seconds; color and glass surfaces second; everything else is polish.

Visual language

- Base: near black background, not pure black, with a subtle grid or circuit line texture behind key screens.
- Accent: a purple to cyan gradient, used sparingly on primary buttons, active states, and the escrow status ring.
- Cards: glassmorphic panels, translucent with blur, for the escrow status and transaction details.
- Type: a geometric sans such as Space Grotesk or Sora for headings, and a monospace face such as JetBrains Mono for wallet addresses and amounts.

Motion

- A pulsing glow on the escrow status ring as it moves between states.
- A brief particle burst when a transaction confirms, small and tasteful, not a firework show.
- An animated scan frame on the QR camera view rather than a static square.
- Shimmer skeletons while waiting on chain confirmation, never a bare spinner.

Flutter animation tooling

flutter_animate for quick declarative transitions. Rive for the QR scan frame and the escrow status ring, since those are the most demoed moments and worth the extra time. AnimatedContainer and AnimatedSwitcher for everyday state changes. Skip Lottie unless an asset already exists, building one from scratch is not a good use of hackathon time.

Technical architecture

Onchain program, Anchor and Rust

PDA seeds:

```
[b"escrow", seller.key, buyer.key, nonce]
```

Account struct:

```
EscrowAccount { seller, buyer, amount, state, created_at }
```

Instructions:

- initialize_escrow, called by the seller to set up the account.
- deposit, called by the buyer to fund it.
- confirm_receipt, called by the buyer to release funds to the seller.

- refund, called by the buyer before confirmation to return funds to themselves.

Flutter app

- solana_kit_mobile_wallet_adapter for wallet connect and signing.
- The solana Dart package for building transactions and reading account state.
- qr_flutter to generate the seller's QR code.
- mobile_scanner for the buyer's camera scan.
- Riverpod for state management, since it pairs well with the async nature of wallet and chain calls.

No Laravel backend for v1. Everything the app needs lives onchain or in local app state. A backend is scope this project does not need yet.

Screens

- Create listing and QR generation.
- Scan and deposit.
- Escrow status.
- Confirm and release.

Build roadmap

- 1 Register the team on solanamobile.radiant.nexus, confirm the submission deadline and repo visibility rules.
- 2 Run `npx solana-mobile@latest doctor` and fix anything it flags before writing code.
- 3 Run a Devnet wallet connect test with `solana_kit_mobile_wallet_adapter`, confirming `authorize` and `signTransactions` work against Solflare.
- 4 Scaffold the Flutter project and add its dependencies.
- 5 Write and deploy the Anchor escrow program to Devnet.
- 6 Build the four core screens.
- 7 Wire each screen's actions to the correct onchain instruction calls.
- 8 Apply the design system once the functional flow works end to end.
- 9 Test the full happy path on a real device or the Solana Mobile emulator.
- 10 Record the demo video, write the pitch, and finalize the GitHub repository.

Explicitly out of scope for v1

- Multi item listings.
- Ratings or reputation system.
- In app chat.
- Dispute arbitration beyond a simple buyer triggered refund.

- A Laravel or any offchain backend.

This document is the working scope for Escrow Pay. Treat the roadmap order as the build order, and treat the non goals list as a guard against scope creep during the thirty day window.